DSD Final Project

Group 1
B12901008 鍾佳聿
B12901137 楊祐宇

Score

Area: 769950.832599 um^2

Synthesis Cycle time: 2.7 ns

Gate simulation Cycle time: 2.7 ns

Qsort: 193798.947 ns

Conv: 34423.347 ns

LFSR_HIST: 722164.647 ns

Score: 3.71e21

Cache Size - CPP Experiment

Cache Size	I-cache Miss Rate	D-cache Miss Rate
16 blocks, 1-way	2.5498% / 3.6880% / 18.4577%	6.7210% / 21.2606% / 37.5730%
16 blocks, 2-way	3.7602% / 4.4479% / 9.6515%	5.0275% / 10.9595% / 37.7028%
32 blocks, 1-way	0.0541% / 0.9164% / 0.0130%	3.8209% / 2.9163% / 36.5347%
32 blocks, 2-way	0.0541% / 0.9164% / 0.0130%	2.7837% / 3.0574% / 36.2103%
64 blocks, 1-way	0.0541% / 0.8941% / 0.0130%	1.7041% / 2.9163% / 35.4964%
64 blocks, 2-way	0.0541% / 0.8941% / 0.0130%	0.9843% / 1.3641% / 34.7826%

Data: Qsort / Conv / LFSR HIST

I cache miss rate saturate at 32block in two cases, since there’s less than 128 instructions.

Cache Size - RTL Experiment

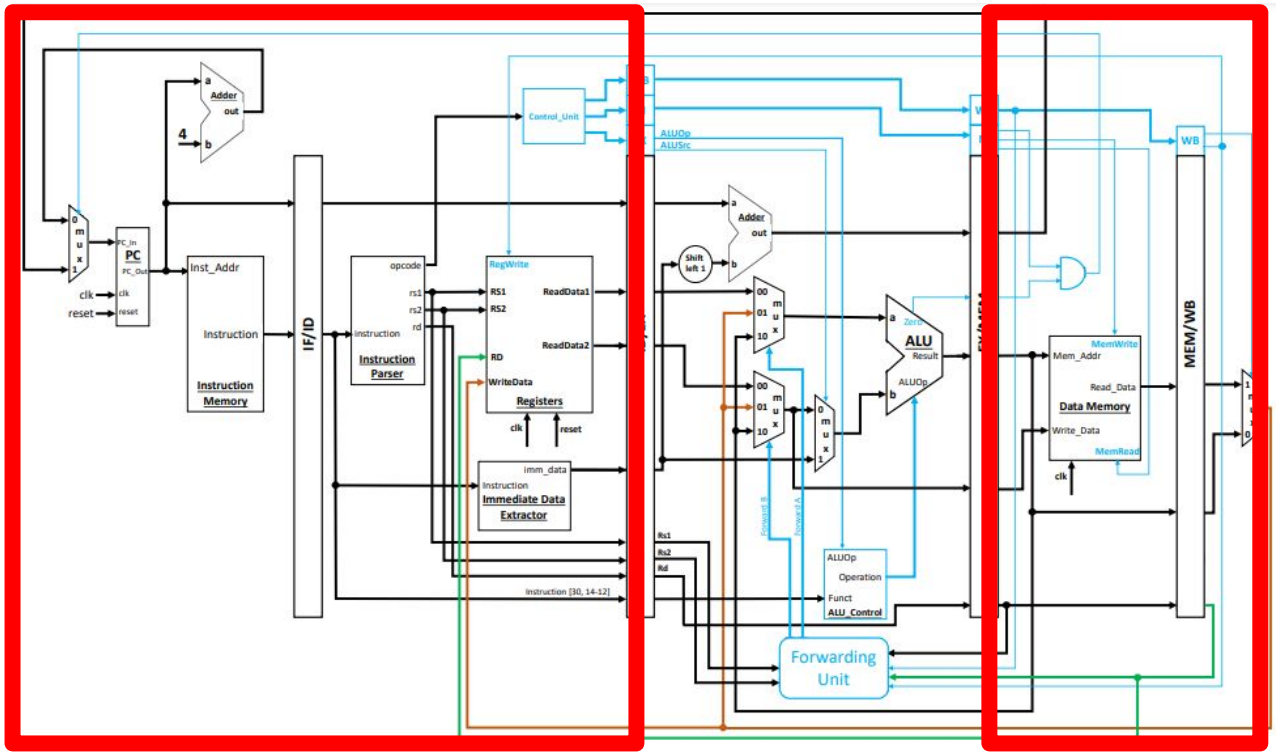
Data:
Qsort / Conv / LFSR HIST , all 1 way cache

Cache Size	Total Cycles	MEM I Stall Cycles	MEM D Stall Cycles
I16 / D16	112201 / 21300 / 1226355	24241 / 1045 / 902177	19538 / 11754 / 18891
I32 / D16	87807 / 20873 / 278220	545 / 566 / 604	19538 / 11754 / 18891
I32 / D32	79718 / 14678 / 277771	545 / 566 / 604	11833 / 5845 / 18449
I32 / D64	73561 / 10965 / 277357	545 / 566 / 604	5945 / 2284 / 18025
I64 / D32	79718 / 14678 / 277771	545 / 566 / 604	11833 / 5845 / 18449
I64 / D64	73561 / 10965 / 277357	545 / 566 / 604	5945 / 2284 / 18025

Compare I16 / D16 and I32 / D16, 32 block I cache decrease LFSR cycle time a lot.
Compare I32 / D32 and I32 / D16, 32 block D cache decrease Qsort, Conv cycle time.

Final Decision: 32 block 1-way I cache + 32 block 2-way D cache

Design - Multiplication Support



When mul in Ex stage, stall other stage for 2 cycle to wait multiplication result to finish.

Branch Prediction - CPP Experiment

Pattern\Algorithm	Always Not Take	Always Taken	BTFNT	3-bit counter
BrPred	37.97%	62.02%	73.42%	68.35%
Qsort	72.44%	27.56%	72.44%	70.48%
Conv	63.58%	36.42%	66.05%	63.58%
LFSR_HIST	85.46%	14.54%	85.46%	85.43%

BTFNT (Backward Taken, Forward Not Taken) predicts backward branches as taken and forward branches as not taken.

Conclusion: Always Not Take achieve high accuracy in the 3 patterns with least hardware cost.

Branch Prediction - Assembly Code Analysis

Example: LFSR_HIST

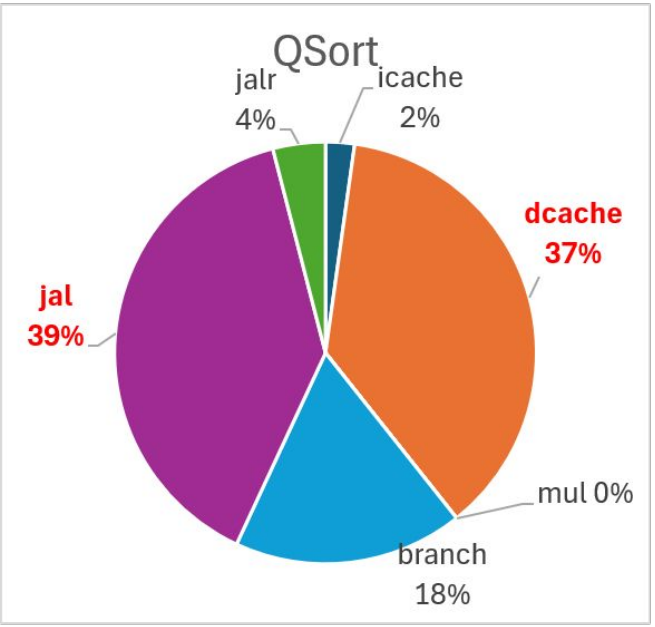
Branch taken or not highly depends on random numbers.
=> Complicated dynamic branch predictors are not suitable.

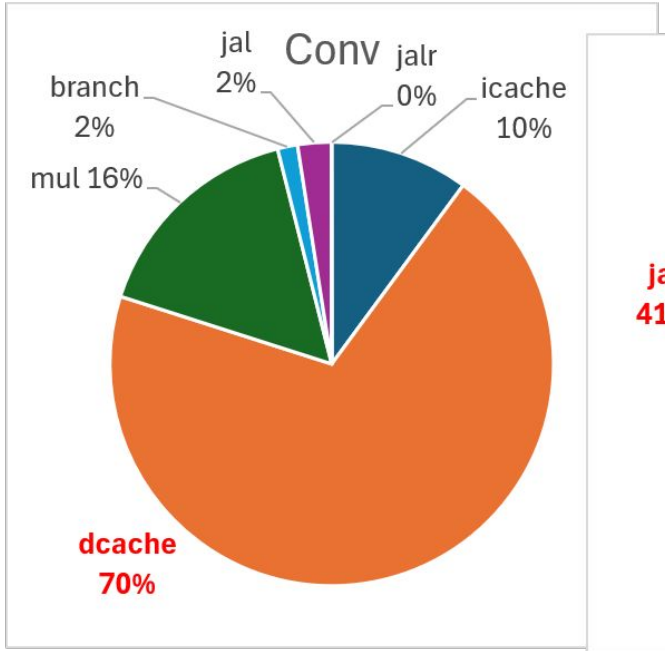
Final Decision: Always Not Taken

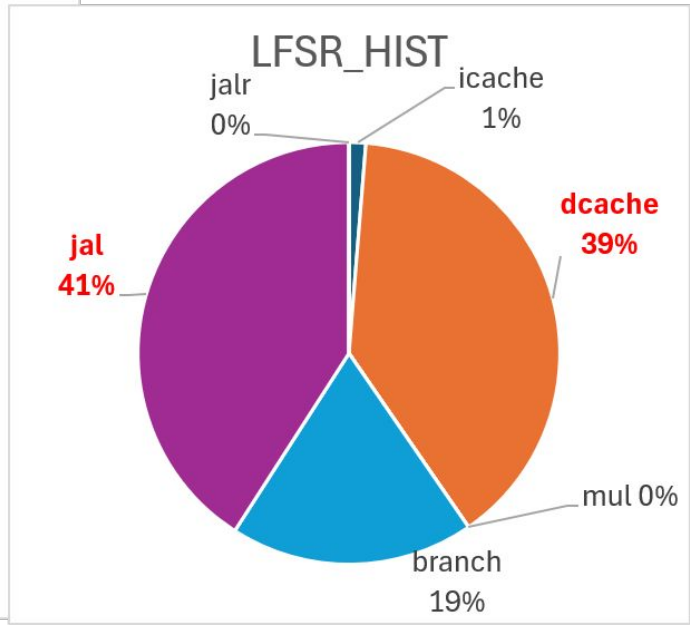
```
# ============================================================
# 第三階段：核心大路口分流 (0x4C ~ 0x70)
# 根據 x11 (當前骰子) 導航到 狀況 A, B, C, D
# ============================================================
0x04C:  beq  x11, x15, COND_A  # 如果 x11 == 0，去狀況 A
0x050:  addi x15, x0, 1
0x054:  beq  x11, x15, COND_A  # 如果 x11 == 1，去狀況 A
0x058:  addi x15, x0, 2
0x05C:  beq  x11, x15, COND_A  # 如果 x11 == 2，去狀況 A
0x060:  addi x15, x0, 3
0x064:  beq  x11, x15, COND_B  # 如果 x11 == 3，去狀況 B
0x068:  addi x15, x0, 4
0x06C:  beq  x11, x15, COND_C  # 如果 x11 == 4，去狀況 C
0x070:  jal  x0,  COND_D       # 若 >= 5，無條件去狀況 D
```

Task Analysis -Stall reason

pattern	stall_total	icache_miss	icache_stall	dcache_miss	dcache_stall	stall_lu	stall_mul	br_occ	br_taken	br_pen	jal_occ	jal_taken	jal_pen	jalr_occ	jalr_taken	jalr_pen	cycle_count
QSort_uncom	20577	28	510	468	8542	4654	0	7354	2027	4054	4804	4494	8988	463	463	926	76253
Conv_uncom	8024	45	818	311	5655	24	1308	162	59	118	148	99	198	2	1	2	12830
LFSR_HIST	31263	31	564	942	17131	0	0	28157	4095	8190	9470	8957	17914	0	0	0	276407







Design - ID stage JAL Resolution

- Move JAL resolution from EX stage to ID stage.
- Penalty:

  2 cycles (resolution at EX, flush IF and ID) -> 1 cycle (resolution at ID, only flush IF)

- Extract a dedicated JAL immediate (fast_jal_imm) directly from instruction bits, bypassing the general immediate generation MUX to shorten the critical path.
- Branch and JALR stays in EX stage since they require additional forwarding unit, which may extend the critical path.

Discarded Optimization Method (Future Works)

- Branch Target Buffer (BTB) for JAL at IF Stage

  Idea: Records JAL instruction PCs and their target addresses to enable immediate redirection during the Instruction Fetch (IF) stage.

  Benefit: Completely eliminates pipeline stalls for recurring JAL instructions, achieving zero-penalty jumps.

  Discarded due to long critical path

- Write Buffer (WB) for D-Cache

  Idea: Temporarily stores evicted dirty blocks from the D-cache, allowing the cache to proceed with memory refills immediately.

  Benefit: Decouples slow memory writebacks from the pipeline critical path, significantly reducing dcache_stall cycles.

  Discarded due to large area

How do we utilize AI?

Generate Baseline Verilog Code

Tell the AI what to do step by step. Check the code between each step.

Step1 : Add Instruction Support, and complicate random latency memory support, and hazard detection, forwarding unit.

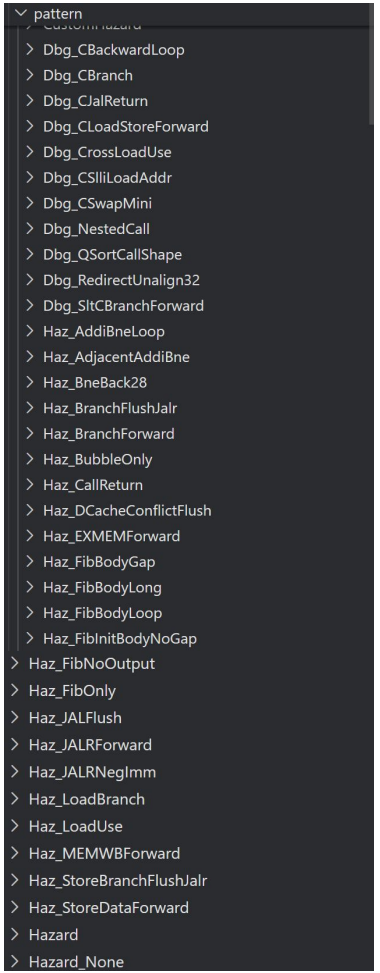
Step2 : Add a baseline cache.

Step3 : Cut pipeline.

After RTL simulation, ask AI to generate test pattern to simulate different scenario for debugging.

Then, PASS!!!

*Highly recommand codex or something like that that can “live” in your IDE, so it can change your file automatically. Or you’ll have to copy and paste a lot.

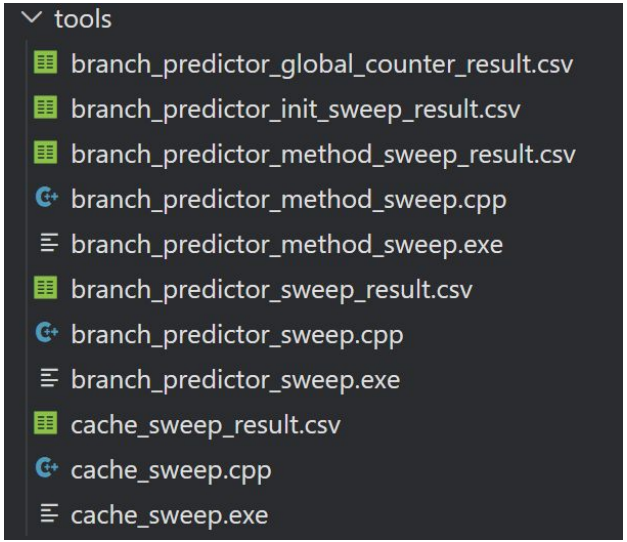


Optimization Experiment

Experiment on cache size, branch prediction……

Ask AI to generate C++ code to read the pattern, and simulate memory or branch prediction unit behavior.

Help us determine direction to improve without writing verilog (and perhaps heavy debugging coming after every modification……)



FAQ : Why not use Python?
C++ is considered faster, and … I'm not the one who have to write it, so why Python?

Some helper file

I’m not good at Linux command (recently learned how to stop a background process, “kill -9”!) and writing Makefile, and I’m lazy opening multiple sessions to run synthesis simultaneously and automatically.

I use AI to generate shell script or modify Makefile, so that I can type less command.

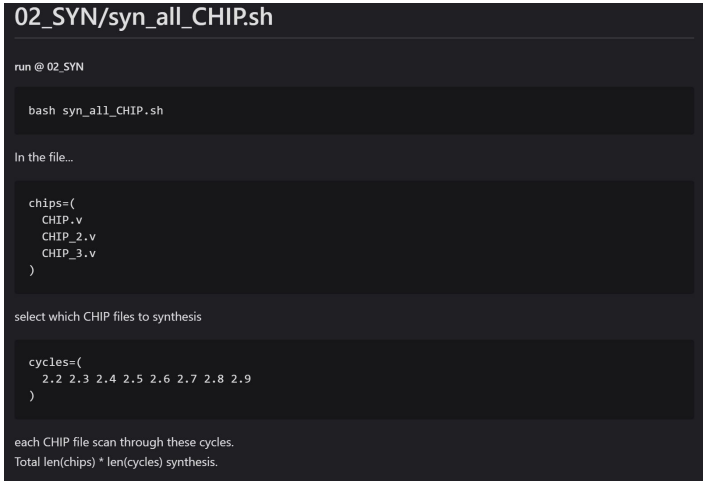
ex.

```
make run_all
```

run all pattern

```
make run3
```

run the 3 graded pattern



screenshot from .md for my teammate.

Use AI but fail:

I tried asking AI to generate slide for checkpoint presentation and this presentation.

But give up since its awful layout and terrible aesthetic taste.

Words too small, and pictures are at weird position…

Not even close to this handmaded elaborated slide!!!

Work Assignment Chart

鍾佳聿	Cache Size Experiment Branch Prediction Experiment Compressed Instruction Extension Multiplication Extension
楊祐宇	Cache Associativity Experiment Assembly Code Analysis ID stage JAL Resolution Synthesis and Optimization

Thanks